vibe-local 入門

aoki-taquan

2026 年 03 月 16 日

目次

1 はじめに 5
1.1 vibe-local とは 5
1.2 利用できる環境 5
1.3 このガイドについて 5
1.4 対象読者 5
1.5 前提知識 5
2 インストール 7
2.1 動作要件 7
2.1.1 モデル別メモリ要件 7
2.2 インストール手順 7
2.2.1 ステップ 1：ターミナルを開く 7
2.2.2 ステップ 2：インストールスクリプトを実行 7
2.2.3 ステップ 3：新しいターミナルで起動 8
2.3 Ollama の手動インストール 8
2.3.1 macOS / Linux 8
2.3.2 Windows 8
2.3.3 モデルの取得 8
2.4 インストールの確認 8
2.5 設定ファイルの場所 8
3 基本操作 9
3.1 起動と終了 9
3.2 対話モードの使い方 9
3.2.1 生成の中断 9
3.3 ワンショットモード 9
3.4 セッションの管理 10
3.4.1 セッション履歴 10
3.4.2 コンテキストの圧縮 10
3.5 プランモード（/plan） 10
3.6 Git チェックポイント 10
3.7 基本的なワークフロー 10
4 組み込みツール 12
4.1 ファイル操作系ツール 12
4.1.1 Read 12
4.1.2 Write 12
4.1.3 Edit 12
4.1.4 NotebookEdit 12
4.2 検索系ツール 12
4.2.1 Glob 12
4.2.2 Grep 12

4.3 実行系ツール 13
4.3.1 Bash 13
4.4 Web 系ツール 13
4.4.1 WebFetch 13
4.4.2 WebSearch 13
4.5 エージェント系ツール 13
4.5.1 SubAgent 13
4.5.2 ParallelAgents 13
4.6 タスク管理ツール 13
4.6.1 TaskCreate / TaskList / TaskGet / TaskUpdate 13
4.7 インタラクション系ツール 14
4.7.1 AskUserQuestion 14
4.8 ツールの権限管理 14
5 高度な機能 15
5.1 プラン/実行モード 15
5.1.1 プランモード 15
5.1.2 実行フェーズへの移行 15
5.2 Git チェックポイント 15
5.2.1 チェックポイントの仕組み 15
5.2.2 活用シーン 15
5.3 スキルシステム 16
5.3.1 スキルの仕組み 16
5.3.2 活用例 16
5.4 MCP（Model Context Protocol）統合 16
5.4.1 MCP で拡張できること 16
5.4.2 MCP サーバーの設定 16
5.5 自動テストループ 17
5.6 ファイルウォッチャー 17
5.7 並行エージェント実行 17
6 モデルと設定 18
6.1 利用可能なモデル 18
6.1.1 推奨モデル 18
6.1.2 サイドカーモデル 18
6.2 モデルの変更 18
6.2.1 起動時に指定 18
6.2.2 設定ファイルで変更 18
6.3 設定ファイル 18
6.3.1 主な設定項目 19
6.4 環境変数 19
6.5 Ollama の設定 19
6.5.1 Ollama サーバーの起動確認 19
6.5.2 インストール済みモデルの確認 19

6.5.3 モデルの追加ダウンロード 19
6.5.4 Ollama のホスト変更 19
6.6 セッションデータの保存場所 20
6.7 ターミナル表示の調整 20
7 Tips と注意事項 21
7.1 安全な使い方 21
7.1.1 AI の提案は必ずレビューする 21
7.1.2 プランモードを活用する 21
7.1.3 Git チェックポイントを積極的に使う 21
7.2 効率的な使い方 21
7.2.1 具体的な指示を与える 21
7.2.2 コンテキストを整理する 21
7.2.3 ワンショットモードを活用する 22
7.2.4 並行エージェントで大規模タスクを処理する 22
7.3 モデル選択のガイド 22
7.3.1 用途別の推奨モデル 22
7.3.2 メモリが少ない環境での工夫 22
7.4 よくある問題と解決策 22
7.4.1 Ollama に接続できない 22
7.4.2 モデルのダウンロードが遅い 22
7.4.3 ターミナル表示が崩れる 23
7.4.4 レスポンスが遅い 23
7.5 vibe-local の活用シーン 23
7.5.1 オフライン環境での開発 23
7.5.2 プライバシーが重要なプロジェクト 23
7.5.3 学習・実験目的 23
7.5.4 チーム内での共有 23

1 はじめに

1.1 vibe-local とは

vibe-local は、完全オフラインで動作する無料の AI コーディングエージェントです。インターネット接続、API キー、クラウドサービスを一切必要とせず、 ローカル環境だけで AI によるコード生成・編集・実行を体験できます。

コアエージェントである `vibe-coder.py` は Python 標準ライブラリのみで書かれた 単一ファイル（約 7,400 行）であり、外部依存関係がありません。

主な特徴は以下の通りです：

- **完全オフライン**：API キー・クラウドサービス・インターネット接続が不要
- **無料・オープンソース**：MIT ライセンスで公開
- **軽量な依存関係**：Python と Ollama のみで動作
- **16 種類の組み込みツール**：ファイル操作、コマンド実行、Web 検索、画像・PDF 読み取りなど
- **MCP 対応**：Model Context Protocol によるツール拡張
- **Git チェックポイント**：作業状態の保存とロールバック
- **プラン/実行モード**：安全な段階的開発フロー

1.2 利用できる環境

vibe-local は以下の環境で動作します：

- **macOS**：Apple Silicon（M1 以降）、16GB 以上の RAM 推奨
- **Windows**：PowerShell / WSL、NVIDIA GPU 推奨
- **Linux**：ターミナル環境、NVIDIA GPU 推奨

1.3 このガイドについて

本書は vibe-local を初めて使う方を対象に、インストールから基本操作、高度な活用法までを解説する入門書です。

1.4 対象読者

- ローカル環境で AI コーディングを試したい方
- クラウド依存なしに AI エージェントを使いたい方
- オフライン環境での開発を必要とする方

1.5 前提知識

- 基本的なターミナル操作（`cd`、`ls` など）
- Python の基礎知識（インストール確認ができる程度）

- Git の基本概念（コミット、ブランチなど）

2 インストール

2.1 動作要件

vibe-local を利用するには以下が必要です：

項目	要件
OS	macOS（Apple Silicon M1+）、Windows 10/11、Linux
Python	Python 3.8 以降
RAM	8GB 以上（モデルによる）
GPU	NVIDIA GPU 推奨（Windows/Linux）
Ollama	別途インストール必要

2.1.1 モデル別メモリ要件

モデル	必要 RAM	用途
`qwen3:1.7b`	8GB	軽量タスク、動作確認
`qwen3:8b`	16GB	標準的な開発作業
`qwen3-coder:30b`	96GB	高精度なコード生成

2.2 インストール手順

2.2.1 ステップ 1：ターミナルを開く

- **Mac**：`Cmd+Space` → 「ターミナル」を検索して起動
- **Windows**：PowerShell を管理者として起動

2.2.2 ステップ 2：インストールスクリプトを実行

2.2.2.1 Mac / Linux / WSL

```
curl -fsSL https://raw.githubusercontent.com/ochyai/vibe-local/main/install.sh | bash
```

2.2.2.2 Windows PowerShell

```
Invoke-Expression (Invoke-RestMethod -Uri https://raw.githubusercontent.com/ochyai/
vibe-local/main/install.ps1)
```

インストールスクリプトは以下を自動的に行います：

- Ollama のインストール（未インストールの場合）
- デフォルトモデル（`qwen3:8b`）のダウンロード
- `vibe-local` コマンドのセットアップ

2.2.3 ステップ 3：新しいターミナルで起動

インストール後、新しいターミナルウィンドウを開いて起動します：

```
vibe-local
```

2.3 Ollama の手動インストール

インストールスクリプトが Ollama を自動インストールしない場合は、手動でインストールします。

2.3.1 macOS / Linux

```
curl -fsSL https://ollama.ai/install.sh | sh
```

2.3.2 Windows

Ollama 公式サイト（https://ollama.ai）からインストーラをダウンロードして実行します。

2.3.3 モデルの取得

```
# 標準モデル（16GB RAM 推奨）
ollama pull qwen3:8b

# 軽量モデル（8GB RAM）
ollama pull qwen3:1.7b

# 高精度モデル（96GB RAM）
ollama pull qwen3-coder:30b
```

2.4 インストールの確認

```
# Ollama の動作確認
ollama list

# vibe-local の起動確認
vibe-local --help
```

2.5 設定ファイルの場所

初回起動後、設定ファイルが以下に作成されます：

```
~/.config/vibe-local/config
```

モデルの変更や詳細設定はこのファイルで行います（詳細は「モデルと設定」章を参照）。

3 基本操作

3.1 起動と終了

プロジェクトディレクトリで `vibe-local` を実行すると対話モードが始まります：

```
cd your-project
vibe-local
```

初回起動時にはプロンプトを直接指定することもできます：

```
vibe-local -p "このプロジェクトの構成を説明して"
```

モデルを明示的に指定して起動する場合：

```
vibe-local --model qwen3:8b
```

終了するには：

- `Ctrl+C` を押す
- `ESC` キーで生成を中断（生成中のみ）

3.2 対話モードの使い方

起動後、プロンプトを自然言語で入力します：

```
> このプロジェクトの構成を説明して
> src/main.py のバグを修正して
> テストを実行して結果を教えて
```

3.2.1 生成の中断

AI が応答を生成中に `ESC` キーを押すと即座に停止できます。長い処理を途中でキャンセルしたい場合に使います。

3.3 ワンショットモード

対話モードに入らず、単発でコマンドを実行できます：

```
# クエリを直接指定
vibe-local -p "スネークゲームを Python で作って"

# パイプと組み合わせ
cat error.log | vibe-local -p "このエラーの原因を分析して"
```

`-p`（`--print`）フラグを使うと結果を標準出力に表示して終了します。

3.4 セッションの管理

3.4.1 セッション履歴

vibe-local はセッションを JSONL 形式で保存します。過去のセッションを参照・再開することで、作業の継続ができます。

3.4.2 コンテキストの圧縮

会話が長くなりコンテキストウィンドウが埋まりそうになると、自動的に古いメッセージが要約・圧縮されます。 これにより長期的な作業セッションでも効率よく動作します。

3.5 プランモード（/plan）

実行前に作業計画を確認できるモードです：

```
/plan
> 認証機能を追加して
```

プランモードでは AI が読み取り専用で分析を行い、実行計画を提示します。 計画を確認してから `/approve` で実際の実行に移行します。

```
/approve
```

実行をキャンセルする場合は新しい指示を入力するだけで元の状態に戻ります。

3.6 Git チェックポイント

作業開始前に Git チェックポイントを作成しておくと、問題が発生した場合にロールバックできます。

チェックポイントは Git の stash 機能をベースにしており、プロジェクトの状態を保存・復元できます。

詳細は「高度な機能」章を参照してください。

3.7 基本的なワークフロー

典型的な開発セッションの流れ：

1. プロジェクトディレクトリに移動する
2. `vibe-local` を起動する
3. `/plan` モードで作業計画を確認する（推奨）
4. 計画を承認して実行する
5. 変更内容をレビューする

6. 問題があればロールバックする

```
cd my-project
vibe-local
# セッション開始後:
# /plan
# > 新しい API エンドポイントを追加して
# /approve
```

4 組み込みツール

vibe-local には 16 種類の組み込みツールが搭載されています。AI がこれらのツールを自律的に使いながらタスクを処理します。

4.1 ファイル操作系ツール

4.1.1 Read

ファイルを読み取るツールです。テキストファイルのほか、 画像、PDF、Jupyter ノートブック（`.ipynb`）にも対応しています。

```
> src/main.py を読んで内容を説明して
> report.pdf の要約を作って
```

4.1.2 Write

新しいファイルを作成します。既存ファイルがある場合は上書きします。

```
> README.md を作成して
```

4.1.3 Edit

既存ファイルの特定の箇所を変更します。 変更差分（diff）をリッチな形式で表示するため、 変更内容を確認しやすくなっています。

```
> src/utils.py の calculate_total 関数にバグがあるので修正して
```

4.1.4 NotebookEdit

Jupyter ノートブックのセルを編集・追加・削除します。データ分析やレポート作成のワークフローに便利です。

4.2 検索系ツール

4.2.1 Glob

ファイル名パターンでファイルを検索します。

```
> プロジェクト内の全テストファイルを調べて
```

4.2.2 Grep

ファイルの内容を正規表現で検索します。

```
> TODO コメントが含まれるファイルを全て探して
```

4.3 実行系ツール

4.3.1 Bash

シェルコマンドを実行します。バックグラウンドジョブにも対応しており、テストの実行、ビルド、パッケージのインストールなどを行えます。

```
> テストを実行して失敗しているケースを教えて
> npm install を実行して
```

4.4 Web 系ツール

4.4.1 WebFetch

指定した URL のコンテンツを取得・解析します。 ドキュメントの参照やサンプルコードの取得に利用できます。

4.4.2 WebSearch

Web 検索を行い、最新情報を取得します。ライブラリのバージョン情報やエラーの解決策調査に役立ちます。

4.5 エージェント系ツール

4.5.1 SubAgent

サブタスクを別のエージェントに委譲します。 複雑な作業を分割して処理する際に使われます。

4.5.2 ParallelAgents

複数のエージェントを並行実行します。独立した複数のタスクを同時に処理することで作業効率が上がります。

例えば、複数のファイルを同時にリファクタリングする場合などに利用されます。

4.6 タスク管理ツール

4.6.1 TaskCreate / TaskList / TaskGet / TaskUpdate

タスクの作成・一覧・取得・更新を行います。 長期的なタスクや複数ステップの作業を管理する際に利用されます。

4.7 インタラクション系ツール

4.7.1 AskUserQuestion

ユーザーに対して確認や選択を求めるプロンプトを表示します。 AI が判断に迷う場合に自動的に呼び出されます。

```
> データベースのテーブルを削除して（AI が確認を求めるケース）
```

4.8 ツールの権限管理

各ツールには安全性に応じた権限ティアが設定されています：

ティア	説明
safe	自動的に実行される安全な操作
ask	実行前にユーザーへの確認が必要
deny	実行が禁止されている操作

`sudo`、`chmod`、長い未知のコマンドなどは `ask` または `deny` に設定されており、 AI が自動的に危険なコマンドを実行することを防ぎます。

5 高度な機能

5.1 プラン/実行モード

プラン/実行モードは、安全に段階的な開発を行うための仕組みです。

5.1.1 プランモード

`/plan` コマンドで有効になる読み取り専用の分析フェーズです。 このモードでは AI がファイルの読み取りや分析を行いますが、 ファイルの変更やコマンドの実行は行いません。

```
/plan
> 認証機能をリファクタリングして
```

AI は変更計画を提示します：

```
計画:
1. auth/login.py を読み取り、現在の実装を確認
2. 重複している検証ロジックを auth/validators.py に分離
3. テストを更新して動作を確認
```

5.1.2 実行フェーズへの移行

計画を確認してから `/approve` で実際の変更を実行します：

```
/approve
```

計画が気に入らない場合は、新しい指示を入力して計画を修正できます。

5.2 Git チェックポイント

Git の stash 機能をベースとしたチェックポイントシステムです。 作業前後の状態を保存し、問題が発生した場合にロールバックできます。

5.2.1 チェックポイントの仕組み

- 作業開始前に現在の状態を自動保存
- エラーや予期しない変更があった場合に復元可能
- 複数のチェックポイントを管理可能

5.2.2 活用シーン

大きな変更を行う前にチェックポイントを作成しておくと安全です：

```
> チェックポイントを作成してから全ファイルをリファクタリングして
```

問題が発生した場合：

```
> 最後のチェックポイントに戻して
```

5.3 スキルシステム

スキルはマークダウンファイルとして定義され、システムプロンプトにドメイン固有の知識を注入します。 コードを変更することなく AI の振る舞いをカスタマイズできます。

5.3.1 スキルの仕組み

スキルファイルを特定のディレクトリに配置すると、vibe-local が起動時に自動的に読み込み、AI のコンテキストに追加します。

5.3.2 活用例

例えば、プロジェクト固有のコーディング規約をスキルとして定義できます：

```
# コーディング規約スキル

このプロジェクトでは以下の規約に従ってください：
- 関数名はスネークケースを使用
- 型ヒントを必ず付ける
- テストカバレッジは 80% 以上を維持
```

5.4 MCP（Model Context Protocol）統合

MCP は外部ツールやサービスを vibe-local に統合するための標準プロトコルです。JSON-RPC 2.0 の stdio ベースで実装されており、 組み込みツール以外の機能を追加できます。

5.4.1 MCP で拡張できること

- データベースへの接続
- 外部 API との連携
- カスタムツールの追加
- 外部サービスのリソース参照

5.4.2 MCP サーバーの設定

設定ファイルで MCP サーバーを指定します：

```
{
  "mcpServers": {
    "my-db-server": {
      "command": "python",
      "args": ["mcp_server.py"]
    }
```

```
  }
}
```

5.5 自動テストループ

編集後に自動でリント・テストを実行する機能です。変更を加えるたびに自動的にテストが実行され、 問題があれば即座にフィードバックが得られます。

```
> テストが全て通るまで自動的に修正を続けて
```

AI は変更→テスト→修正のサイクルを自律的に繰り返します。

5.6 ファイルウォッチャー

ファイルの変更をポーリングで検出する機能です。ファイルが変更されるとリアクティブなワークフローが起動します。

開発中に外部ツールやエディタでファイルを変更した場合でも、 AI がその変更を検知して自動的に対応できます。

5.7 並行エージェント実行

`ParallelAgents` ツールを使うことで、 複数のエージェントを同時に実行できます。

独立した複数のタスク（例：異なるモジュールのリファクタリング）を 並行処理することで、大規模な作業を効率化できます。

```
> frontend と backend を並行してテストして
```

6 モデルと設定

6.1 利用可能なモデル

vibe-local は Ollama 経由でローカルモデルを利用します。 `localhost:11434` に対して OpenAI Chat API 互換のリクエストを送信します。

6.1.1 推奨モデル

モデル名	必要 RAM	用途	特徴
`qwen3-coder:30b`	96GB	高精度コード生成	最高品質、大規模プロジェクト向け
`qwen3:8b`	16GB	標準開発作業	バランス重視、推奨
`qwen3:1.7b`	8GB	軽量タスク	低スペック環境向け

6.1.2 サイドカーモデル

vibe-local は、権限確認や初期化などの軽量タスクに「サイドカーモデル」を使います。 メインモデルより軽いモデルが自動選択されるため、リソースの節約になります。

6.2 モデルの変更

6.2.1 起動時に指定

```
vibe-local --model qwen3:8b
```

6.2.2 設定ファイルで変更

```
# 設定ファイルを開く
nano ~/.config/vibe-local/config
```

設定ファイルの例：

```
model=qwen3:8b
sidecar_model=qwen3:1.7b
```

6.3 設定ファイル

設定ファイルは `~/.config/vibe-local/config` に保存されています。

6.3.1 主な設定項目

設定項目	説明
`model`	使用するメインモデル名
`sidecar_model`	サイドカータスク用モデル名
`ollama_host`	Ollama のホスト URL（デフォルト: localhost:11434）

6.4 環境変数

デバッグや動作調整に使える環境変数：

環境変数	説明
`VIBE_LOCAL_DEBUG=1`	デバッグモードを有効化（詳細ログを出力）
`VIBE_DEBUG_TUI=1`	TUI（ターミナル UI）のデバッグ情報を表示
`VIBE_NO_SCROLL=1`	スクロール領域機能を無効化（ターミナル互換性問題の解決）

使い方の例：

```
VIBE_LOCAL_DEBUG=1 vibe-local
```

6.5 Ollama の設定

6.5.1 Ollama サーバーの起動確認

```
ollama serve
```

通常は自動起動しますが、問題がある場合は手動で起動します。

6.5.2 インストール済みモデルの確認

```
ollama list
```

6.5.3 モデルの追加ダウンロード

```
ollama pull qwen3:8b
ollama pull qwen3:1.7b
```

6.5.4 Ollama のホスト変更

デフォルト以外のホストで Ollama を動かす場合、設定ファイルで変更します：

```
ollama_host=http://192.168.1.100:11434
```

6.6 セッションデータの保存場所

セッション履歴は JSONL 形式で保存されます：

```
~/.config/vibe-local/sessions/
```

古いセッションデータを削除することで、ディスク容量を節約できます。

6.7 ターミナル表示の調整

vibe-local は DECSTBM（スクロール領域制御）を使って 固定フッターを表示します。ターミナルの互換性問題が発生した場合は：

```
VIBE_NO_SCROLL=1 vibe-local
```

これにより、スクロール領域機能が無効になります。

7 Tips と注意事項

7.1 安全な使い方

7.1.1 AI の提案は必ずレビューする

AI は完璧ではありません。特に以下のコマンドを含む提案は 実行前に必ず内容を確認してください：

- `sudo` を含むコマンド
- `chmod`、`chown` などのパーミッション変更
- `rm -rf` などの削除操作
- 長い、見慣れないコマンド

7.1.2 プランモードを活用する

大きな変更を行う前には必ず `/plan` モードで計画を確認しましょう：

```
/plan
> プロジェクト全体をリファクタリングして
```

計画が意図通りかを確認してから `/approve` で実行します。

7.1.3 Git チェックポイントを積極的に使う

重要な変更の前には Git チェックポイントを作成しておくことを推奨します。 これにより、問題が発生した場合に簡単に元の状態に戻せます。

7.2 効率的な使い方

7.2.1 具体的な指示を与える

曖昧な指示より具体的な指示の方が精度が上がります：

```
# 曖昧な指示
> コードを直して

# 具体的な指示
> src/auth.py の login 関数で、パスワードが空文字の場合に
> ValueError を発生させるバリデーションを追加して
```

7.2.2 コンテキストを整理する

長いセッションでは、コンテキストが肥大化して精度が低下することがあります。定期的に不要なコンテキストを整理し、新しいセッションを開始することを検討してください。

7.2.3 ワンショットモードを活用する

単発のタスクには対話モードより `-p` フラグのワンショットモードが効率的です：

```
# エラーログの分析
cat error.log | vibe-local -p "エラーの原因と解決策を提案して"

# コードレビュー
vibe-local -p "src/ ディレクトリのコードをレビューして問題点を列挙して"
```

7.2.4 並行エージェントで大規模タスクを処理する

独立した複数のタスクは並行実行させることで効率化できます：

```
> frontend のテストと backend のテストを同時に実行して
```

7.3 モデル選択のガイド

7.3.1 用途別の推奨モデル

- **単純な質問・小規模変更**：`qwen3:1.7b`（高速、低メモリ）
- **通常の開発作業**：`qwen3:8b`（バランス重視）
- **複雑なアーキテクチャ設計**：`qwen3-coder:30b`（高精度）

7.3.2 メモリが少ない環境での工夫

RAM が 8GB 程度の場合は `qwen3:1.7b` を使用し、 タスクを細かく分割して処理するのが効果的です。

7.4 よくある問題と解決策

7.4.1 Ollama に接続できない

```
# Ollama の起動状態を確認
ollama serve

# プロセスが動いているか確認
ps aux | grep ollama
```

7.4.2 モデルのダウンロードが遅い

`qwen3:8b` は数 GB のサイズがあります。 安定したネットワーク環境でダウンロードし、完了後はオフラインでも使えます。

7.4.3 ターミナル表示が崩れる

スクロール領域機能との互換性問題の場合：

```
VIBE_NO_SCROLL=1 vibe-local
```

7.4.4 レスポンスが遅い

- より小さいモデルに切り替える
- 他のアプリケーションを閉じてメモリを確保する
- GPU が使われているか確認する（Linux/Windows）

7.5 vibe-local の活用シーン

7.5.1 オフライン環境での開発

インターネット接続がない環境（飛行機内、セキュリティ制限のある環境など）でも AI によるコーディング支援が使えます。

7.5.2 プライバシーが重要なプロジェクト

コードがクラウドに送信されないため、 機密性の高いコードベースでも安全に使用できます。

7.5.3 学習・実験目的

API キーや費用なしに AI コーディングエージェントを体験できるため、 教育や研究目的での利用に最適です。

7.5.4 チーム内での共有

同じ Ollama サーバーを複数人で共有することで、 チーム全員が同じモデルを使った AI 支援を受けられます。